

LLD Case Study 8: In-Memory API Rate Limiter

Target Patterns: Strategy Pattern, Encapsulation, State Synchronization
Key Engineering Focus: High-throughput rate-limiting algorithms: Token Bucket, Leaky Bucket, and Sliding Window Log with thread-safe client isolation.

1. Problem Statement & Functional Requirements

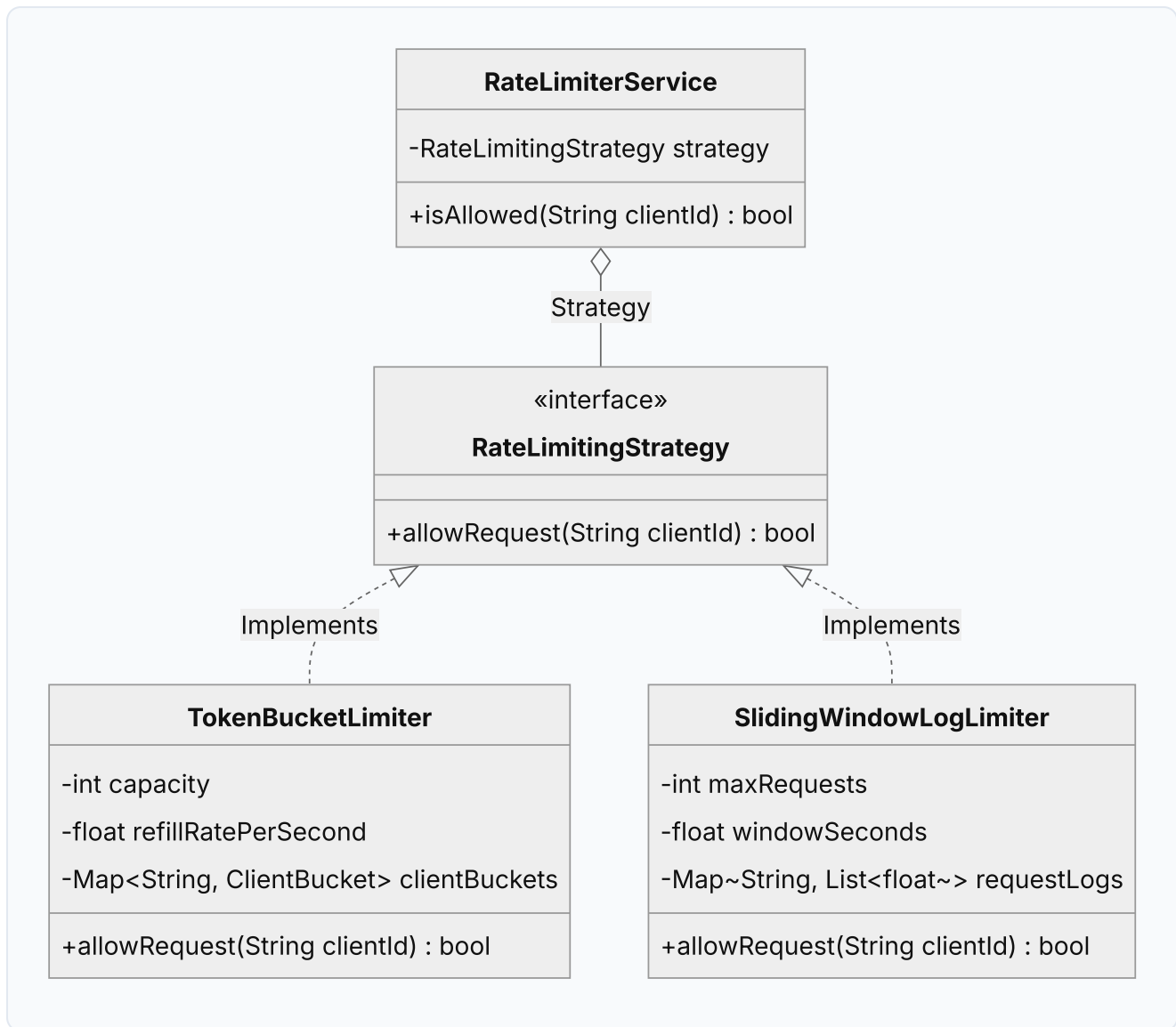
Design an API rate limiter capable of protecting backend microservices from Denial-of-Service (DoS) attacks, brute-force requests, and resource starvation.

Requirements:

- 1. **Pluggable Algorithms (Strategy Pattern):** Easily interchange between **Token Bucket**, **Leaky Bucket**, and **Sliding Window Log**.
- 2. **Client Isolation:** Maintain independent rate limits per Client ID / IP address.
- 3. **Thread-Safety:** Atomic token consumption under high concurrent request volume.
- 4. **Instant Evaluation:** Must execute in under 1 millisecond ($O(1)$ time complexity).

2. Algorithm Comparison & Trade-Offs

Algorithm	Memory Consumption	Burst Handling	Accuracy	Use Case
Token Bucket	$O(1)$ per client	Supports bursts up to bucket capacity	High	General API Gateways (Stripe, GitHub)
Leaky Bucket	$O(\text{Capacity})$ buffer	Enforces smooth, fixed outflow rate	High	Egress traffic shaping
Sliding Window Log	$O(N)$ requests in window	Accurate without boundary resets	100% Exact	Strict financial APIs



3. Production-Grade Python Implementation